Noted Nexus Router v0.04 AI-Assisted Technical Specification

Noted Nexus Router v0.04 - AI-Assisted Systems Technical Specification

Status: technical-specification sweep artifact
Base archive: noted_nexus_router_pokemon_merge_v0.03.zip
Prepared archive: noted_nexus_router_ai_spec_bearing_v0.04.zip
Edition: v0.04, three-sweep specification
Primary claim: the archive is not a delivery wrapper; the archive is a load-bearing development object.

Executive abstract

Noted Nexus Router v0.04 is specified as a local-first knowledge workspace in which Noted acts as the host archive, Nexus OS acts as the router/kernel, and block applications act as sovereign tools mounted through an event contract. The Gameboy-style shell is not a cosmetic skin. It is the operator-facing console that keeps the system playable while the underlying graph of notes, prompts, agents, creatures, forges, engines, and online actions grows.

The system is also a demonstration of AI-assisted development. The zip is treated as the unit of trust. Conversation history is disposable. Source code, planning documents, load-bearing contracts, schemas, registries, footers, verification logs, and architectural breadcrumbs travel together. A future AI worker must be able to pick up the archive without this conversation and know what to do, what not to touch, how to verify it, and where to place the next block.

This specification is written in three sweeps:

1. **Sweep I - Whitepaper frame.** Defines the operating thesis: Noted remembers, Nexus routes, Nostr speaks, Gameboy shell makes it playable, AI acts through capability gates.
2. **Sweep II - Load-bearing code contract.** Pins the concrete seams in the current app: Noted routes, Nexus iframe boundary, Nexus managed-block handshake, block-client boot protocol, action envelope, Nostr translation, UI patch, and agent action model.
3. **Sweep III - Build demonstration.** Turns the spec into an archive-resident development protocol: planning packet, registries, schemas, bridge types, verification gates, and block-by-block implementation sequence.

Core mental model

```
Noted
└── Nexus Router route (/nexus-router)
    └── Nexus OS kernel / Gameboy shell
        ├── managed blocks
        │   ├── Nexus Agent
```

```
│   ├── Prompt Studio v3
│   ├── Pokémon/Eidolon engines
│   ├── Lattice and world tools
│   └── social / wallet / system blocks
├── event language
├── capability registry
├── action broker
├── Nostr bridge
└── UI mutation bridge
```

The correct architecture is not “Noted has a sidebar link for every tool.” The correct architecture is “Noted has a durable host route for Nexus, and Nexus owns the inner routing language.” Direct Noted routes may exist as compatibility fallbacks for development, but they are not the product direction.

Why this is a technical spec and a whitepaper at once

A normal whitepaper argues a system should exist. A normal technical spec says how to implement it. This document does both because the artifact itself is the argument. Every architectural claim must land as one of the following:

- a path in the archive,
- a schema,
- a TypeScript type,
- a block manifest,
- a registry entry,
- a verification check,
- a build block,
- a load-bearing code excerpt,
- or a deferred decision with a revisit point.

That makes the zip a document, the document a scaffold, and the scaffold a runnable development protocol.

Sweep I - Whitepaper Frame: The Archive as Operating System Seed

1. The system being specified

Noted Nexus Router is a local-first personal operating surface for thinking, writing, prompting, building, playing, syncing, and delegating. It is not merely a note app. It is a layered archive in which different classes of tools live at different authority levels.

Noted is the host. It owns the user’s canonical workspace: notes, prompts, scraps, projects, canvases, libraries, shelves, and other durable local objects. It already has a React/Vite/Electron shell, IndexedDB-backed workspace behavior, route structure, and local application framing. It should remain stable and boring.

Nexus is the router. It is where dynamic blocks, experimental tools, Gameboy-shell interactions, agents, forges, social modules, wallet modules, engines, and future Nostr connections belong. It is the membrane between stable personal archive and volatile tool ecosystem.

Nostr is the external event fabric. It should not be embedded separately into every block. The router translates internal Nexus events into Nostr events only when policy, identity, capability, signing, and relay rules say that should happen.

The AI agent is not a god-mode script runner. It is an actor inside the router. It proposes, requests, transforms, files, dispatches, and annotates. It can act online only through an action broker. It can change UX only through reversible UI patches. It can self-evolve only through reviewable diffs, tests, lineage, and rollback.

2. The Gameboy shell as a serious systems decision

The Gameboy style is important because this system can become cognitively huge. The shell gives the user a consistent interaction grammar:

- **Screen:** what is currently being played or operated.
- **Menu:** where routes and blocks become selectable cartridges.
- **Inventory:** objects the user owns: notes, prompts, agents, Eidolons, keys, workflows, receipts.
- **Link cable:** Nostr and other external event bridges.
- **Save slot:** Noted archive state and export snapshots.
- **Mission log:** agent proposals, receipts, failures, approvals, and history.
- **Evolution lab:** controlled self-improvement of agents, prompts, blocks, and UI layouts.

The visual metaphor controls complexity. Without it, the architecture risks becoming a developer dashboard. With it, everything can still be deep, but every operation has an understandable home.

3. The zip as load-bearing object

The archive is not an attachment. The archive is the continuity surface. The current session can vanish. The next AI may be a different model. The human may not remember why a path exists. Therefore the zip must carry its own operating memory.

The v0.04 archive should contain:

```
PROJECT_NOTES.md
CONTEXT.md
BUILD_BLOCKS.md
HANDOFF.md
BLOCK_PROMPT_BB-00.md
specs/
  NOTED_NEXUS_AI_ASSISTED_TECHNICAL_SPEC_v0.04.md
  SWEEP_1_WHITEPAPER_THESIS.md
  SWEEP_2_LOAD_BEARING_CONTRACTS.md
```

```
  SWEEP_3_IMPLEMENTATION_DEMONSTRATION.md
  LOAD_BEARING_CODE_EXCERPTS.md
  AI_ASSISTED_DEVELOPMENT_DEMO.md
  SECURITY_AND_CAPABILITY_MODEL.md
  NOSTR_ROUTING_SPEC.md
  AGENT_ACTION_AND_UI_MUTATION_SPEC.md
public/nexus/registry/
  block-registry.v0.04.json
  channel-registry.v0.04.json
  capability-registry.v0.04.json
  action-registry.v0.04.json
  nostr-kind-registry.v0.04.json
public/nexus/bridges/
  nexus-event-envelope.schema.json
  agent-action.schema.json
  ui-patch.schema.json
  nostr-translation.schema.json
src/bridges/
  nexusBridgeTypes.ts
  nexusActionTypes.ts
  nostrBridgeTypes.ts
```

The point is not to add paperwork. The point is to make the app buildable by agents without losing architectural intent.

4. The language every app speaks

Every block must eventually speak the Nexus Event Envelope. Some legacy blocks can remain iframed leaf apps, but managed blocks speak through the router.

Canonical event flow:

```
block emits event
→ Nexus validates source, channel, payload, and capability
→ Nexus routes event locally or to Noted bridge
→ optional: Nexus translates event to Nostr
→ optional: action broker proposes external operation
→ user approves where required
→ executor runs operation
→ receipt returns to Nexus
→ Noted archives receipt if durable
```

The same language covers notes, prompt snapshots, Forge results, agent plans, UI patches, Pokémon/Eidolon state, Nostr posts, relay sync, online actions, and self-evolution proposals.

5. Noted, Nexus, Nostr, agent: authority boundaries

Layer	Authority	Should do	Should not do
Noted host	Stable personal archive	Store canonical user objects, expose bridge, run app shell	Become the router for every experimental block
Nexus router	Inner OS / event kernel	Mount blocks, validate channels, route events, broker actions	Write directly to Noted without a bridge receipt
Gameboy shell	Operator console	Make complex actions navigable and playful	Hide dangerous actions behind aesthetics
Blocks	Sovereign tools	Work locally, emit declared events, consume declared events	Invent private sync or mutate host state directly
Nostr bridge	External event adapter	Translate selected events to signed relay events	Become each block's private network library
Agent	Delegated actor	Plan, propose, organize, transform, act through capabilities	Silently mutate data, publish, spend, sign, or rewrite itself

6. Self-evolution stance

The agent self-evolves, but not by editing itself in secret. The self-evolution loop is:

observe friction
→ propose improvement
→ classify mutation ring
→ generate diff or workflow
→ run tests / static checks
→ show preview
→ get approval if required
→ apply through controlled executor
→ write receipt
→ monitor behavior
→ rollback if bad
→ preserve lineage

This is the difference between an evolving system and an unsafe autonomous mutation machine.

7. The AI-assisted development thesis

AI-assisted development does not mean “ask an AI to code.” In this system it means:

- architecture is captured as executable constraints,
- build blocks are small enough to verify,
- every archive carries the current project brain,
- every changed code file explains its scope and load-bearing surfaces,
- every future AI worker is forced to load Context / Scope / Verify before changing anything,
- risky regions are ring-classified,
- and no block is trusted until the gate passes.

The app itself becomes both product and method. It is a tool for organizing the user and a case study in making AI-generated software survive more than one chat window.

Sweep II - Load-Bearing Code Contracts

This sweep names the current code seams that future implementation must preserve or intentionally supersede. These excerpts are not decorative. They are the present anchors for the technical specification.

1. Noted route contract

Current code: src/App.tsx

```
<div className="flex-1 flex flex-col min-w-0">
 <TopBar />
 <div className="flex-1 flex flex-col min-h-0 overflow-hidden">
  <main className="flex-1 min-h-0 overflow-y-auto overflow-x-hidden">
   <div key={routeKey} className="n-route-transition h-full">
    <Routes>
     <Route path="/" element={<Navigate to="/nexus-router" replace />} />
     <Route path="/inbox" element={<Inbox />} />
     <Route path="/writing/*" element={<WritingStudio />} />
     <Route path="/notes/*" element={<NotesStudio />} />
     <Route path="/poetry/*" element={<PoetryStudio />} />
     <Route path="/longform/*" element={<LongformStudio />} />
     <Route path="/app-design/*" element={<AppDesignStudio />} />
     <Route path="/projects" element={<Projects />} />
     <Route path="/prompts" element={<PromptStudio />} />
     <Route path="/canvas" element={<Canvas />} />
     <Route path="/nexus-router" element={<NexusRouterStudio />} />
     <Route path="/nexus-agent" element={<NexusAgentStudio />} />
     <Route path="/prompt-studio-v3" element={<PromptStudioV3 />} />
```

```
          <Route path="/atlas" element={<Atlas />} />
          <Route path="/library" element={<Library />} />
          <Route path="/shelf/*" element={<Shelf />} />
          <Route path="/scraps" element={<ScrapsStudio />} />
          <Route path="/settings" element={<Settings />} />
          <Route path="*" element={<Navigate to="/canvas" replace />} />
        </Routes>
```

Spec interpretation:

- / redirects to /nexus-router.
- /nexus-router is the front door for the inner OS.
- /nexus-agent and /prompt-studio-v3 remain development and fallback routes.
- Future app additions should generally become Nexus blocks, not new Noted routes.

Load-bearing marker: @load-bearing: /nexus-router is the stable Noted-to-Nexus host route.

2. Noted-to-Nexus iframe seam

Current code: src/studios/nexusRouter/NexusRouterStudio.tsx

```
const NEXUS_ROUTER_SRC = './nexus/os/Nexus_OS.html'

export function NexusRouterStudio(): JSX.Element {
  function openStandalone() {
    window.open(NEXUS_ROUTER_SRC, '_blank', 'noopener,noreferrer')
  }

  return (
    <section className="h-full min-h-0 flex flex-col bg-bg" data-test="nexus-router-
studio">
      <header className="shrink-0 border-b border-line bg-surface/80 px-4 py-3 flex items-
center justify-between gap-3">
        <div className="min-w-0">
          <div className="text-[10px] uppercase tracking-[0.22em] text-ink-faint">Kernel
router</div>
          <h1 className="text-sm font-semibold text-ink mt-0.5">Nexus Router</h1>
          <p className="text-[11px] text-ink-soft mt-1 leading-relaxed">
            Noted is the host. Nexus OS is the router/kernel. Nexus Agent, Prompt Studio v3,
Pokémon/Eidolon engines, and other HTML blocks launch inside the Nexus desktop.
          </p>
        </div>
        <div className="shrink-0 flex items-center gap-2">
          <a
```

```
          href={NEXUS_ROUTER_SRC}
          target="_blank"
          rel="noreferrer"
          className="rounded border border-line px-3 py-1.5 text-[11px] text-ink-soft
hover:text-ink hover:bg-surface-3 transition-colors"
        >
          Open tab
        </a>
        <button
          type="button"
          onClick={openStandalone}
          className="rounded border border-line bg-surface-2 px-3 py-1.5 text-[11px] text-
ink-soft hover:text-ink hover:bg-surface-3 transition-colors"
        >
          Pop out
        </button>
      </div>
    </header>
    <div className="flex-1 min-h-0 bg-black">
      <iframe
        title="Nexus Router"
        src={NEXUS_ROUTER_SRC}
        className="block h-full w-full border-0 bg-black"
        data-test="nexus-router-iframe"
        referrerPolicy="no-referrer"
        sandbox="allow-scripts allow-forms allow-modals allow-popups allow-popups-to-
escape-sandbox allow-downloads allow-same-origin"
      />
    </div>
  </section>
 )
}
```

Spec interpretation:

This iframe is the host bridge seam. Do not replace it with a deep React integration yet. The next correct move is to attach a typed postMessage bridge around this seam, not to dissolve the iframe boundary.

The iframe sandbox currently allows scripts, forms, modals, popups, downloads, and same-origin. This is permissive because the integrated Nexus OS blocks need browser-native behavior. Future hardening should be capability-based rather than simply removing flags until blocks fail.

Load-bearing marker: @load-bearing: Nexus remains iframe-mounted until the bridge contract is implemented and tested.

3. Nexus kernel settings contract

Current code: public/nexus/os/Nexus_OS.html

```
const SETTINGS = Object.freeze({
  watchdogMs:          12000,
  maxRetries:          1,     // only 1 retry (2 total attempts) — 3-flash bug was maxRetries:2
  stableMountResetMs:  5000,
  reaperIntervalMs:    5000,
  pongDeadlineMs:      20000,  // was 9000 — much more generous
  firstPongGraceMs:    90000, // 90s grace for blocks that have never PONGed yet
  pingGraceMs:         25000,
  pingIntervalFloorMs: 4000,
  payloadLimitBytes:   131072,  // raised from 8192 — apps need more headroom
  payloadMaxDepth:     10,
  payloadMaxNodes:     1024,
  payloadMaxKeysPerObject: 256,
  payloadMaxArrayLength: 512,
  dataLimiterCapacity: 100,
  dataLimiterRefillPerSec: 50,
  controlLimiterCapacity: 10,
  controlLimiterRefillPerSec: 5,
  maxRateStrikes:      8,
  maxManifestChannels: 48,
  maxSubscriptionsPerBlock: 48,
  maxChannelNameLen:   96,
  maxManifestBytes:    4096,
```

Spec interpretation:

The Nexus kernel already has a process model: watchdogs, retry caps, payload bounds, manifest bounds, subscriptions, rate limiting, and channel-length constraints. The technical spec formalizes this as the router runtime contract.

The payload and manifest limits are not random constants. They define a safety membrane. Increase only with test coverage and explicit rationale.

Load-bearing marker: @load-bearing: managed block runtime limits are protocol safety limits, not UI tuning values.

4. Managed block handshake

Current code: public/nexus/os/Nexus_OS.html

```
  }

  // Protocol handlers
  function handleDeclare(block, msg) {
    pushFeed("INFO", `${block.id} ← DECLARE received (state was ${block.state})`);
    if (block.state !== "LOADING") { violation(block, "unexpected DECLARE"); return; }
    try {
      block.manifest = normalizeManifest(msg.manifest);
      UNDOABLE_CHANNELS.set(block.id, new Set(block.manifest.undoable || []));
      if (msg.app && typeof msg.app === "object") {
        block.appMeta = {
          title:       String(msg.app.title       || block.id).slice(0, 64),
          icon:        String(msg.app.icon        || " ▪ ").slice(0, 4),
          description: String(msg.app.description || "").slice(0, 128),
          visible:     msg.app.visible !== false
        };
      }
      block.state = "DECLARED";
      block.mountNonce = crypto.randomUUID();
      noteAccepted(block, "DECLARE");
      pushFeed("INFO", `${block.id} declared [${block.manifest.emits.join(",")||"∅"}] / [${block.manifest.consumes.join(",")||"∅"}]`);
      block.port.postMessage({ type: "MOUNT_CHALLENGE", nonce: block.mountNonce });
      if (block.appMeta.visible && !block.windowEl) {
        Shell.createWindow(block);
      }
      Shell.scheduleRender();
    } catch (err) {
      pushFeed("BAD", `${block.id} bad DECLARE: ${err.message}`);
      evictBlock(block.id, "bad DECLARE");
    }
  }

  function handleSub(block, msg) {
    if (block.state !== "DECLARED" && block.state !== "MOUNTED") { violation(block, "SUB wrong state"); return; }
    if (typeof msg.channel !== "string") { violation(block, "SUB missing channel"); return; }
    if (!block.manifest.consumes.includes(msg.channel)) { violation(block, `undeclared consume ${msg.channel}`); return; }
    if (block.desiredSubscriptions.has(msg.channel)) { noteAccepted(block, "SUB_NOOP"); return; }
    if (block.desiredSubscriptions.size >= SETTINGS.maxSubscriptionsPerBlock) { violation(block, "sub cap"); return; }
```

```
    block.desiredSubscriptions.add(msg.channel);
    if (block.state === "MOUNTED") activateSubscriptions(block);
    persistBlockState(block);
    noteAccepted(block, "SUB");
    pushFeed("INFO", `${block.id} sub ${msg.channel}`);
    Shell.scheduleRender();
  }

  function handleMountAck(block, msg) {
    pushFeed("INFO", `${block.id} ← MOUNT_ACK received`);
    if (block.state !== "DECLARED") { violation(block, "unexpected MOUNT_ACK");
return; }
    if (!msg || msg.nonce !== block.mountNonce) { violation(block, "invalid MOUNT_ACK
nonce"); return; }
    block.state = "MOUNTED";
    block.mountNonce = null;
    block.lastPongAt = Date.now();
    block.outstandingPings.clear();
    // Notify the block that the handshake is complete. Managed blocks that gate
    // kernel-IPC on this signal (e.g. wallet _ipcMounted) depend on this message.
    try {
      block.port.postMessage({ type: "MOUNTED" });
    } catch (err) {
      pushFeed("BAD", `${block.id} MOUNTED ack failed: ${err.message}`);
    }
    activateSubscriptions(block);
    noteAccepted(block, "MOUNT_ACK");

    // Create window for visible managed blocks on successful handshake completion.
    // Legacy blocks create their window in iframe.onload — managed blocks must do it here,
```

Spec interpretation:

Managed blocks follow the lifecycle:

BOOT with MessagePort
→ DECLARE manifest/app metadata
→ MOUNT_CHALLENGE nonce
→ MOUNT_ACK nonce
→ MOUNTED
→ SUB / EMIT / PING / PONG

The handshake is the kernel's trust surface. A block cannot simply emit whatever it wants. It declares capabilities first. The kernel normalizes and bounds the manifest. Undeclared subscriptions are rejected. Undoable channels must also be emitted.

Load-bearing marker: @load-bearing: DECLARE -> MOUNT_CHALLENGE -> MOUNT_ACK -> MOUNTED is the managed-block mount contract.

5. Shared block client adapter

Current code: public/nexus/os/engines/nexus-block-client.js

```
  }
  const set = handlers.get(channel);
  if (set) {
   for (const fn of Array.from(set)) {
    try { fn(p, src, raw); }
    catch (err) { console.error('[nexus-block-client] handler failed', channel, err); }
   }
  }
  if (typeof onMessage === 'function') {
   try { onMessage({channel, payload: p, src, raw}); }
   catch (err) { console.error('[nexus-block-client] onMessage failed', channel, err); }
  }
 }

 function onBoot(e){
  if (!e.data || e.data.type !== 'BOOT') return;
  port = e.ports && e.ports[0];
  blockId = e.data.blockId || blockId;
  if (!port) return;

  post({
   type: 'DECLARE',
   manifest: declared.manifest,
   app: declared.app || undefined
  });

  port.onmessage = ev => {
   const msg = ev.data;
   if (!msg || typeof msg.type !== 'string') return;
   if (msg.type === 'MOUNT_CHALLENGE') {
    post({type:'MOUNT_ACK', nonce: msg.nonce});
   } else if (msg.type === 'MOUNTED') {
    mounted = true;
    for (const channel of requestedSubs) post({type:'SUB', channel});
    while (queuedEmits.length) post(queuedEmits.shift());
    readyResolve(api);
   } else if (msg.type === 'PING') {
```

```
      post({type:'PONG', nonce: msg.nonce});
    } else if (msg.type === 'MSG') {
      handleMsg(msg.channel, msg.payload || {}, msg.src, msg);
    }
  };
  if (typeof port.start === 'function') port.start();
}

global.addEventListener('message', onBoot);

const api = {
  emit,
  request,
  handle,
  subscribe,
  unsubscribe(channel, handler){
    const set = handlers.get(channel);
    if (set && handler) set.delete(handler);
  },
  ready,
  get mounted(){ return mounted; },
  get blockId(){ return blockId; },
  _debug: { pending, handlers, requestHandlers, requestedSubs, queuedEmits }
};
return api;
}

 global.NexusBlockClient = { bootBlock, equivocationProof, commitReveal, _stable:
stable, _sha256Hex: sha256Hex, _randomSaltHex: randomSaltHex };
if (typeof module !== 'undefined' && module.exports) module.exports =
global.NexusBlockClient;
```

Spec interpretation:

The block client is the adapter every managed block should eventually use. It gates subscription declarations, queues emits until mounted, handles PING/PONG, exposes ready, and provides request/response helpers.

Legacy blocks can remain, but the spec should classify them as legacyIframe or unmanagedHtml. Future reliable blocks should use NexusBlockClient.bootBlock() or an equivalent versioned successor.

Load-bearing marker: @load-bearing: NexusBlockClient.bootBlock is the canonical managed block adapter for future blocks.

6. Canonical event envelope

```
export type NexusEventEnvelope = {
  id: string
  createdAt: string
  source: NexusActorRef
  target?: NexusActorRef
  kind: NexusEventKind
  intent?: string
  capability?: string
  channel: string
  tags: NexusTag[]
  refs: NexusRef[]
  payload: unknown
  policy: NexusPolicy
  receipt?: ActionReceipt
}
```

Required design decisions:

- source is always explicit.
- channel is the router-facing protocol lane.
- kind is the semantic type.
- capability declares what permission is needed.
- policy.requiresApproval determines whether the action can run immediately.
- receipt is appended after execution, not guessed before it.

7. Action broker contract

The action broker is the only sanctioned path from agent intent to effectful action.

agent proposes action
→ router validates schema
→ capability registry checks actor rights
→ policy decides approval threshold
→ preview is generated
→ user approves if required
→ executor runs
→ receipt is saved
→ optional Nostr translation occurs

Actions are divided into categories:

Category	Examples	Approval default
Local organization	tag note, create prompt draft,	auto or ask-once

Category	Examples	Approval default
	route inbox item	
UI mutation	add button, change layout, create workspace view	preview + approve
Nostr read	fetch relay state, sync public event	allow after relay config
Nostr write	publish note, sync app data, send reply	approve unless user policy allows
Online action	webhook, GitHub issue, email draft, calendar event	approve
Self-evolution	patch code, change prompt, alter agent policy	high-friction review

8. UI mutation contract

UI changes must be patches, not hidden direct DOM edits.

```ts
export type NexusUiPatch = {
  id: string
  target: 'gameboy-shell' | 'nexus-router' | 'noted-host' | string
  scope: 'session' | 'workspace' | 'profile'
  operation: 'layout' | 'theme' | 'widget' | 'menu' | 'route' | 'macro'
  rationale: string
  preview: string
  diff?: unknown
  reversible: true
  requiresApproval: boolean
}
```

Agent-driven UI change is allowed only if it can be previewed, reverted, and traced to a receipt.

9. Nostr translation contract

Nostr is the external event fabric. It should receive only selected, policy-approved Nexus events.

NexusEventEnvelope
→ NostrTranslationPlan
→ signing request
→ relay publish
→ publish receipt
→ Noted archive receipt

NIP-01 supplies the basic event and relay message model. NIP-78 supplies a natural target for arbitrary app-specific data using kind:30078. NIP-46 supplies remote signing so private keys

can remain outside arbitrary blocks. NIP-65 supplies relay-list metadata for read/write relay preferences.

The router should expose a nostr.publish.requested action. Blocks should not import their own Nostr SDKs unless a future block explicitly authorizes that exception.

10. Self-evolution contract

Self-evolution is implemented as an action family, not as direct source writes:

agent.evolution.observation.created
agent.evolution.patch.proposed
agent.evolution.patch.reviewed
agent.evolution.patch.approved
agent.evolution.patch.applied
agent.evolution.patch.verified
agent.evolution.patch.rolled_back

Any mutation touching ring 0 or ring 1 surfaces requires explicit review. A self-evolving agent may generate a patch, but it may not silently apply a patch that changes:

- event names,
- source-of-truth order,
- storage ownership,
- identity/signing policy,
- bridge security,
- app boot order,
- Nexus block handshake,
- Noted database schema,
- or installer behavior.

Sweep III - Implementation Demonstration and Build Packet

This sweep turns the spec into a work program that a coding AI can execute. The point is not to finish every feature in one burst. The point is to make the archive ready for disciplined forward motion.

1. What v0.04 adds to the archive

The v0.04 spec-bearing archive adds three classes of material:

1. **Human-readable doctrine:** the whitepaper and technical spec.
2. **Machine-readable contracts:** TypeScript bridge types, JSON schemas, and registries.
3. **AI-worker continuity files:** project notes, context, build blocks, handoff, and the first block prompt.

This means a future coding worker can open the zip and start from the archive itself, not from this chat.

2. First implementation milestone

The first real implementation should not be Nostr publish or online action execution. The first milestone should be a local bridge loop:

Prompt Studio v3 emits prompt.snapshot.created
→ Nexus Router receives it
→ Nexus forwards it to Noted bridge
→ Noted creates or updates a Prompt record
→ Noted returns a receipt
→ Nexus displays confirmation
→ receipt is archived

This proves the whole architecture locally without requiring relay keys, signing, remote permissions, or online actions.

3. Build block sequence

ID	Name	Primary outcome
BB-00	Spec-bearing scaffold	Add bridge types, schemas, registries, planning packet, no behavior mutation
BB-01	Noted bridge listener	Add typed host-side postMessage listener and receipt channel
BB-02	Nexus host adapter	Add Nexus-side adapter that can target Noted host events
BB-03	Prompt Studio managed shim	Wrap Prompt Studio v3 emission into managed event language
BB-04	Prompt snapshot import	Convert prompt.snapshot.created into Noted prompt records
BB-05	Action broker dry run	Add proposed action queue, preview, approval state, receipts
BB-06	UI patch preview	Add reversible UI patch model for Gameboy shell only
BB-07	Agent action proposal	Let Nexus Agent propose local organization actions, no external execution
BB-08	Nostr identity/signer config	Add NIP-46 oriented signer

ID	Name	Primary outcome
		configuration and relay list policy
BB-09	Nostr read bridge	Read selected app events from configured relays
BB-10	Nostr write bridge	Publish approved app-specific events and receipts
BB-11	Online action adapters	Add webhook/GitHub/email/calendar adapters as approval-gated mocks first
BB-12	Self-evolution lab	Add patch proposal, diff, test command, approval, rollback receipts
BB-13	Gameboy UX integration	Make actions, inventory, mission log, link cable, and evolution lab navigable
BB-14	Package and audit	Produce final verified archive/artifact with docs and clean run path

4. Verification philosophy

Every block must verify three things:

1. **The app still builds.** npm run typecheck and npm run build must pass.
2. **The contract did not drift.** Grep or script checks must confirm no silent changes to @load-bearing lines, no unauthorized new persistence, no unauthorized network calls, and no SDK imports outside approved blocks.
3. **The behavior works.** A short manual or automated smoke test must prove the block's event path.

5. The demonstration narrative

The product demo should eventually show this path:

User writes rough prompt in Noted
→ Send to Nexus / Prompt Studio
→ Prompt Studio analyzes and forges improved prompt
→ Agent proposes filing result into project
→ User approves
→ Noted stores it as prompt artifact
→ Nexus mission log records receipt
→ Optional: publish encrypted or app-specific sync event to Nostr
→ Another device pulls event and restores the prompt artifact

This demo proves the architecture as product and method:

- Noted remembers.
- Nexus routes.
- Prompt Studio transforms.
- Agent organizes.
- User approves.
- Nostr transports.
- Archive preserves.
- Verification keeps the system from drifting.

6. Security stance for online actions

Online actions are not browser automation free-for-all. The system must represent online actions as adapter-specific capabilities.

online.fetchUrl.preview
online.fetchUrl.execute
nostr.publish.preview
nostr.publish.execute
github.issue.createDraft
github.issue.submit
email.draft.create
calendar.event.createDraft

High-risk actions require confirmation every time until the user changes policy. Credentials must be stored only through explicitly specified blocks. Blocks cannot sneak in SDKs or auth headers. Any new provider adapter must have a mock mode and a receipt mode before real execution.

7. The UX mutation demonstration

A safe UI mutation demo:

Agent observes: user repeatedly sends notes to Forge.
Agent proposes: add “Forge this” action to note cards.
Router classifies: UI patch, workspace scope, reversible.
User previews: button placement and behavior.
User approves.
Gameboy shell / Noted host applies patch.
Receipt records patch ID and revert path.

Unsafe version:

Agent directly rewrites JSX, changes note card behavior, and stores hidden preference.

The spec forbids the unsafe version.

8. The self-evolution demonstration

A safe self-evolution demo:

Agent finds that Prompt Studio snapshots do not include analysis score.
Agent proposes schema extension: add optional `analysisScore`.
Router classifies as ring1 because event payload shape changes.
Spec shows diff and affected consumers.
Tests prove backward compatibility.
User approves.
Patch applies.
Receipt records old schema, new schema, test results, rollback path.

This is the principle: self-evolution is not magic. It is disciplined patch flow with stronger UX.

9. What must not happen next

Do not add five more iframes directly to Noted. Do not add Nostr separately to Prompt Studio, Agent, Lattice, and Pokémon engines. Do not let the agent bypass the router. Do not let UI mutation become hidden DOM mutation. Do not let self-evolution happen outside receipts.

The next phase is small, local, and provable: Prompt Studio snapshot to Noted prompt through Nexus Router.

Sweep II-B - Full Event Language Detail

1. Event taxonomy

The router event language uses two axes: semantic kind and transport channel. The semantic kind describes what happened or is requested. The channel describes where the router should deliver it. These should not be collapsed. A future event may keep the same kind but move across a different channel as the architecture evolves.

kind = meaning
channel = routing lane
capability = authority required
policy = approval and reversibility rules
receipt = proof of outcome

Event kind families

Family	Examples	Meaning
diagnostic.*	diagnostic.ping, diagnostic.receipt	Safe bridge proving and health checks
prompt.*	prompt.snapshot.created, prompt.snapshot.imported	Prompt artifacts and transformations

Family	Examples	Meaning
agent.*	agent.action.proposed, agent.evolution.patch.proposed	Agent planning and controlled action
ui.*	ui.patch.proposed, ui.patch.applied	Reversible UX adaptation
nostr.*	nostr.publish.requested, nostr.publish.receipted	External relay sync and publication
eidolon.*	eidolon.creature.snapshot, eidolon.battle.receipt	Game object and engine state
mission.*	mission.log.entry, mission.receipt.created	Operator narrative and audit trail
system.*	system.capability.denied, system.block.mounted	Kernel and capability events

The event family is not only naming. It implies default risk. diagnostic.* is safe by default. nostr.*, online.*, agent.evolution.*, and persistent noted.write operations are not.

2. Envelope validation stages

A router or host bridge should validate an envelope in stages. Each stage returns an explicit receipt or rejection reason.

shape check
→ source check
→ channel declaration check
→ capability check
→ payload schema check
→ policy check
→ dedupe/idempotency check
→ executor dispatch
→ receipt

Shape check

The bridge confirms that the incoming message is an object, has type NEXUS_HOST_BRIDGE or kernel equivalent, and contains an envelope with id, source, kind, channel, payload, and policy.

Source check

The source actor must match the origin channel. A child iframe may claim to be a block, but it may not claim to be Noted host or the user. The router maps MessagePort identity, iframe identity, block id, and declared manifest to actor identity.

Channel declaration check

Managed blocks must declare the channels they emit and consume. A block that did not declare prompt.snapshot.created cannot suddenly emit it. This is already the spirit of the Nexus managed-block handshake; the v0.04 bridge extends that discipline to host interactions.

Capability check

Capability is the semantic permission. A message can be syntactically valid and still lack authority. For example, a Prompt Studio block may emit prompt.snapshot.created, but it may not publish to Nostr or mutate Noted database state without the router and host bridge.

Payload schema check

Payload schemas start permissive, then become stricter as blocks are migrated. The first proof target is deliberately narrow: diagnostic ping, receipt, prompt snapshot created, and prompt import requested.

Policy check

Policy decides if the event can execute now, needs approval, or must be rejected. A low-risk diagnostic ping can execute. A noted.write prompt import may require approval until the user trusts the workflow. A Nostr publish event requires signer and approval policy.

Dedupe and idempotency

Every effectful event should carry an id. Re-sending the same id should return the existing receipt if possible. This matters for relay retries, agent retries, and browser refresh.

3. Receipt structure

Receipts are small but critical. They are the durable confirmation surface. They should be usable by humans and by future agents.

```
type ReceiptCore = {
  id: string
  actionId?: string
  envelopeId?: string
  ok: boolean
  actor: NexusActorRef
  executor: NexusActorRef
  capability?: string
  target?: NexusRef
  summary: string
  createdAt: string
  reversible: boolean
  rollbackRef?: NexusRef
  error?: string
```

```
  refs: NexusRef[]
}
```

Receipts must not overclaim. If an action only created a draft, the receipt says draft. If a Nostr event was signed but not published, the receipt says signed-not-published. If three relays accepted and two failed, the receipt carries per-relay status.

4. Local proof loop: Prompt Studio to Noted

The first local bridge proof uses Prompt Studio because it is already a prompt-analysis and Forge environment and it already contains the conceptual shape of snapshots and prompt evolution.

Target event:

```
{
  "kind": "prompt.snapshot.created",
  "channel": "prompt.snapshot.created",
  "source": {"kind":"block","id":"prompt-studio-v3"},
  "payload": {
    "title": "Generated or user-provided title",
    "body": "Prompt text",
    "format": "RAW|CPL|XML|COSTAR|RISEN|...",
    "analysis": {
      "score": 0,
      "tokens": 0,
      "components": []
    },
    "origin": "manual|pre-run|forge|adversarial|transform"
  },
  "policy": {
    "requiresApproval": true,
    "reversible": true,
    "risk": "medium",
    "capability": "noted.write"
  }
}
```

The event does not directly write to Noted. It requests import. The router converts it into prompt.snapshot.import.requested toward the host bridge. The host bridge writes only in BB-04 after explicit implementation.

Sweep II-C - Router / Host Bridge API

1. Host bridge responsibilities

The Noted host bridge has four responsibilities:

1. Receive typed messages from the Nexus iframe.
2. Reject malformed or unauthorized messages.
3. Dispatch approved host actions to Noted services.
4. Return receipts to Nexus.

It does not route internal block-to-block events. That remains Nexus territory. It does not become a Nostr client. That remains bridge territory inside Nexus with signer policy.

2. Host bridge non-responsibilities

The host bridge must not:

- expose raw workspace context to all blocks,
- allow arbitrary query access to IndexedDB,
- accept source claims without validation,
- execute online actions,
- sign Nostr events,
- execute code strings,
- or become a backdoor around the action broker.

3. Message format

Host-facing messages use a wrapper to distinguish them from incidental browser messages.

```
type HostBridgeTransportMessage = {
  type: 'NEXUS_HOST_BRIDGE'
  envelope: NexusEventEnvelope
}
```

Host receipts use the same wrapper with receipt payload:

```
type HostBridgeReceiptMessage = {
  type: 'NEXUS_HOST_RECEIPT'
  receipt: ActionReceipt
  inReplyTo: string
}
```

BB-01 should implement only enough for diagnostics. BB-04 should add prompt import. No block should get broad host read/write access just because the listener exists.

4. Bridge listener placement

The listener belongs at the Nexus Router studio seam because that component owns the iframe. It may delegate to a hook or helper, but the ownership remains clear.

NexusRouterStudio
→ iframe ref
→ useNexusHostBridge(iframeRef, handlers)
→ validate incoming messages
→ return receipts with iframe.contentWindow.postMessage

The implementation should avoid global app-wide message listeners with broad permissions. If a global listener is used, it must still verify event source against the iframe window.

5. Host action registry

Host actions are finite. The initial registry should be tiny:

diagnostic.ping
prompt.import.preview
prompt.import.execute (future)

The action registry grows by block. Every action has:

- id,
- capability,
- payload schema,
- risk,
- approval policy,
- executor,
- receipt shape,
- test path.

6. Error semantics

The bridge should reject explicitly. Silent failure is forbidden.

Error	Meaning
invalid_message_shape	Wrapper or envelope missing required fields
invalid_source	Message not from the Nexus iframe or actor mismatch
unsupported_action	Valid envelope but no executor registered
capability_denied	Actor lacks capability
approval_required	Action cannot run until approved
execution_failed	Executor threw or returned failure

Sweep II-D - Registry Discipline

1. Why registries exist

Registries are the bridge between prose and code. A technical spec says “blocks declare events.” A registry says which blocks, which events, which actions, which capabilities, and which Nostr kinds exist now.

The registries do not replace runtime validation. They make validation finite.

2. Block registry

The block registry names every app-like block the router knows about. Each entry should eventually include:

```
{
  "id": "prompt-studio-v3",
  "title": "Prompt Studio v3",
  "path": "/nexus/os/blocks/apps/prompt-studio-v3.html",
  "mode": "legacyIframe",
  "future": "managedBlock",
  "owner": "core",
  "risk": "medium",
  "storage": "localStorage:ps3*",
  "emits": [],
  "consumes": []
}
```

mode matters. Legacy iframes are tolerated; managed blocks are trusted only through the managed-block handshake.

3. Channel registry

The channel registry prevents event-name drift. It should answer:

- who owns the channel,
- who may emit it,
- who may consume it,
- what payload schema applies,
- what capability is required,
- what block introduced it,
- whether it is undoable.

4. Capability registry

The capability registry separates action possibility from action permission. A block may technically be able to call postMessage; that does not mean it has noted.write.

5. Nostr kind registry

The Nostr registry maps app intentions to Nostr event kinds. It exists so future workers do not casually invent custom kinds or misuse public-note kinds for private app state.

Sweep II-E - Storage Ownership Model

1. Existing reality

The current app has multiple storage surfaces:

- Noted IndexedDB workspace,
- Nexus OS internal storage behavior,
- block-specific localStorage keys,
- embedded quine/export data in some standalone apps,
- game/engine persistence inside included subprojects,
- future Nostr relay state.

The spec must not pretend this is already unified. The correct move is staged migration.

2. Storage authority tiers

Tier	Owner	Examples	Migration posture
Canonical archive	Noted	notes, prompts, projects, receipts	preserve and bridge into
Router state	Nexus	mounted blocks, mission log, block registry	keep router-owned
Block-local legacy	individual apps	Prompt Studio ps3, Agent local state	wrap then migrate selectively
External sync	Nostr relays	app data events, receipts	bridge only after signer policy
Generated packages	zip/export/quines	self-contained snapshots	preserve as artifacts

3. Do not unify too early

Early unification would be risky. Prompt Studio, Nexus Agent, and game engines were developed as standalone systems. Their storage assumptions may carry hidden behavior. The bridge should first move explicit artifacts through explicit events. Only later should legacy state be migrated.

4. Canonical object candidates

Objects likely to become Noted canonical records:

- prompt snapshots,
- Forge results,
- agent action receipts,
- mission log entries,
- Nostr publish receipts,
- user-approved UI patches,
- selected game objects / Eidolon snapshots,
- project summaries generated by the agent.

Objects that may remain Nexus or block-local:

- transient UI state,
- battle animation state,
- ephemeral agent scratch reasoning,
- unmanaged block settings,
- temporary remote relay cache.

Sweep II-F - Agent Operating Model

1. Agent roles

The system should distinguish at least five agent roles:

Role	Scope	Example
Organizer	local archive organization	route scraps into projects
Promptsmith	prompt improvement	forge and score prompts
Operator	action proposal	draft Nostr post or GitHub issue
Cartographer	cross-app linking	link notes, prompts, agents, game objects
Evolver	system improvement	propose patch or UX adaptation

The same underlying AI may play multiple roles, but the router should model role because capability and approval differ.

2. Agent memory

Agent memory should be explicit artifacts, not hidden chat state. Candidate memory classes:

- user preference notes,
- project rules,
- rejected proposals,
- approved workflows,

- UI patches,
- successful action receipts,
- failed action receipts,
- self-evolution lineage.

3. Agent control surfaces

Agent control must enter through constrained surfaces:

agent.organize.request
agent.action.proposed
agent.ui.patch.proposed
agent.evolution.patch.proposed
agent.nostr.publish.draft

There should be no generic agent.execute(anything) channel. Generic execution is where safety dies.

4. Online action adapters

Adapters should be boring. Each adapter exposes:

- preview(input)
- validate(input)
- execute(input, approval)
- receipt(result)

Examples:

nostr.publish
webhook.send
github.issue.create
email.draft.create
calendar.event.create
url.fetch.summary

All real online actions should have mock mode first. The mock mode proves the action broker, preview, approval, and receipt before real credentials appear.

Sweep II-G - Self-Evolution Model

1. What self-evolution means here

Self-evolution is the system improving its own prompts, workflows, UI, schemas, and code through controlled patches. It is not an unbounded recursive agent.

2. Evolution artifact types

Artifact	Mutation risk	Example
Prompt rule	low/medium	change Forge review instruction
Workflow macro	medium	add “Forge this note” flow
UI patch	medium/high	add note-card action
Schema change	high	add field to prompt snapshot payload
Source patch	high/critical	change bridge listener behavior
Kernel patch	critical	change Nexus handshake

3. Evolution lab UI

The Gameboy shell should eventually expose an Evolution Lab with:

- observed issue,
- proposed change,
- ring classification,
- affected files/schemas,
- diff preview,
- test plan,
- approval control,
- apply button,
- rollback button,
- receipt.

4. Evolution receipts

Evolution receipts should include:

- patch id,
- actor,
- rationale,
- files changed,
- ring classification,
- tests run,
- approval record,
- before/after digests,
- rollback instructions,
- follow-up observations.

Sweep III-B - AI-Assisted Development Method Embedded in the App

1. Why the uploaded workflow matters

The uploaded workflow documents define a disciplined AI build loop: a strategist creates planning docs, a worker implements one block per archive, a gate verifies, and the archive moves forward. This app adopts that as its own development method.

The spec-bearing archive carries:

- the project brain,
- the current build sequence,
- the next block prompt,
- the verification rules,
- the handoff state,
- the architecture spec,
- the code excerpts,
- and the machine-readable bridge contracts.

2. Worker session behavior

A future worker begins by reading the archive, not the conversation. The first worker response must state:

Context: files loaded, active block, load-bearing constraints.
Scope: files to change, files not to change, why.
Verify: commands and manual checks.

Then it implements only the active block. After the block, it updates the archive, not just the code.

3. Verification gate as firewall

The gate is not a suggestion. It prevents a single hallucinated implementation from becoming the base of future work.

For this project, gate checks include:

- archive extracts cleanly,
- no unexpected credentials,
- npm ci succeeds,
- npm run typecheck succeeds,
- npm run build succeeds,
- no unauthorized fetch, SDK import, telemetry, or persistence is introduced,
- planning docs status is updated,
- footers are present,
- load-bearing markers are not silently changed,

- active block done criteria are satisfied.

4. FFD / CPL influence

The FFD/CPL protocol contributes the ring model, mutation posture, breadcrumb discipline, and the idea that code artifacts can carry operational memory. For this project, FFD hooks become most important around:

- Nexus OS boot and managed block handshake,
- host bridge listener,
- action broker,
- Nostr signer boundary,
- Noted database write bridge,
- self-evolution patch executor,
- installer scripts.

5. Ring classification for this app

Region	Ring	Why
Noted database schema	0	canonical local archive
Nexus managed block handshake	0	trust and lifecycle boundary
installer scripts	0/1	user launch and data path assumptions
host bridge listener	1	cross-boundary protocol surface
action broker	1	permissioned effect execution
Nostr signer bridge	1/0	identity and signing boundary
block UI	2	replaceable presentation
Gameboy theme	2	presentation unless it hides policy
Prompt scoring heuristics	2	leaf logic until persisted as canonical policy

6. Breadcrumb requirements

A future worker must add a breadcrumb when:

- an event name changes,
- a payload schema changes,
- a block moves from legacy iframe to managed block,
- a capability default changes,
- a Nostr kind mapping changes,

- a UI patch changes default user workflow,
- a self-evolution rule is accepted or rejected.

Sweep III-C - Acceptance Tests and Smoke Scripts

1. BB-01 acceptance

The BB-01 bridge listener is accepted when:

- Nexus iframe still loads.
- Invalid bridge messages are ignored or rejected with explicit receipt.
- A diagnostic ping returns a diagnostic receipt.
- No Noted database mutation occurs.
- No Nostr or online code is introduced.
- Typecheck/build pass.

2. BB-04 acceptance

Prompt import is accepted when:

- Prompt Studio snapshot event arrives at Nexus.
- Nexus sends import request to host.
- Host validates payload.
- Host creates Noted prompt record or draft.
- Host returns receipt with created object ref.
- Duplicate envelope id is idempotent.
- Failure produces rejected receipt without partial write.

3. BB-10 acceptance

Nostr write bridge is accepted when:

- User has relay policy.
- User has signer configuration.
- Publish request previews the exact event.
- User approves.
- Event is signed through signer path.
- Relay attempts are recorded separately.
- Receipt returns with event id and relay results.
- Failed relay does not claim success.

4. BB-12 acceptance

Self-evolution lab is accepted when:

- Agent can propose patch but cannot apply it silently.

- Diff is shown.
- Ring classification is shown.
- Tests are listed before approval.
- Patch application writes receipt.
- Rollback path is present.
- Ring0 changes require explicit human review.

Final Synthesis

This technical spec is intentionally heavy because the project is no longer a single app. It is an operating surface, a router, a game shell, an agent environment, a Nostr bridge candidate, and an AI-assisted development demonstration. The risk is not that the idea is too small. The risk is that the idea becomes too powerful without enough structure.

The structure is now explicit:

Archive carries memory.
Noted carries canon.
Nexus carries routing.
Blocks carry tools.
Agent carries intent.
Broker carries permission.
Receipts carry proof.
Nostr carries selected events.
Gameboy shell carries usability.
Verification gate carries trust.

The next code block should be modest: host bridge listener. That is how the massive system becomes real without becoming reckless.

Appendices

Appendix A - v0.04 contract inventory

Contract	Location	Ring	Review
/nexus-router host route	src/App.tsx	1	ring1
Nexus iframe seam	src/studios/nexusRouter/NexusRouterStudio.tsx	1	ring1
Nexus managed block handshake	public/nexus/os/Nexus_OS.html	0	ring0

Contract	Location	Ring	Review
Nexus block adapter	public/nexus/os/engines/nexus-block-client.js	1	ring1
Event envelope	src/bridges/nexusBridgeTypes.ts	1	ring1
Action broker model	src/bridges/nexusActionTypes.ts	1	ring1
Nostr translation	src/bridges/nostrBridgeTypes.ts	1	ring1
JSON schemas	public/nexus/bridges/*.schema.json	1	ring1
Registries	public/nexus/registry/*.json	1	ring1
Planning docs	root markdown files	0	human/ring0

Appendix B - Definition of done for this spec-bearing sweep

- The archive extracts cleanly.
- The spec lives inside the archive.
- The planning packet lives inside the archive.
- Bridge type files compile under TypeScript.
- JSON schemas and registries are parseable.
- Existing app behavior is not intentionally changed.
- npm run typecheck passes.
- npm run build passes.
- A future AI worker can identify BB-01 as the next implementation block.

Appendix C - Non-goals for this sweep

This sweep does not implement Nostr publishing, real online actions, prompt import, UI mutation application, remote signing, or agent self-evolution. It makes those changes implementable without turning the next session into architectural guesswork.